endava

UNIVERSITATEA DIN BUCUREȘTI
VIRTUTE ET SAPIENTIA

PROGRAMARE ORIENTATA PE OBIECTE II

CURS 10

AGENDA CURS 10

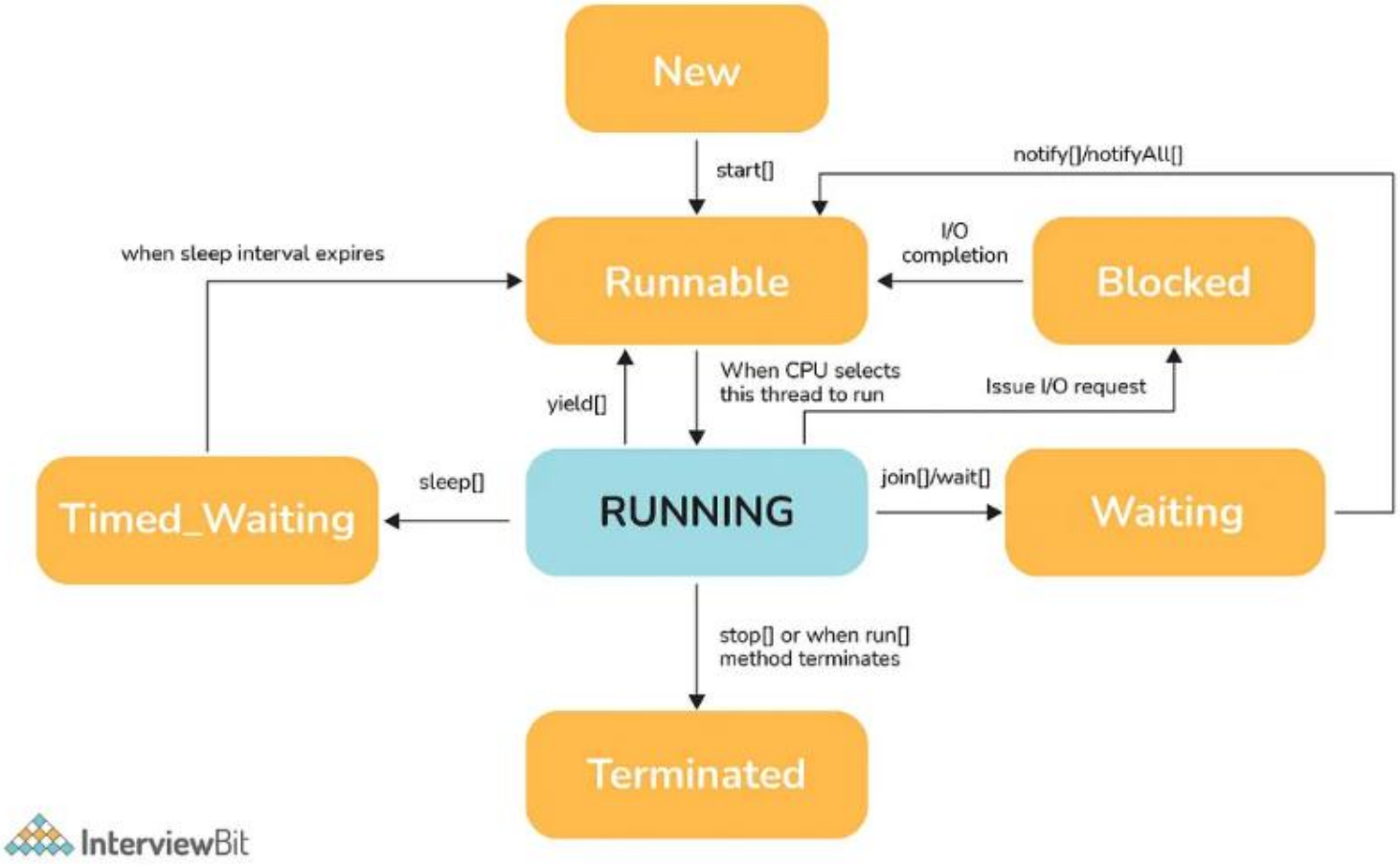
1. Fire de executie
2. Crearea firelor de executie
3. Probleme in context multithreaded
4. Sincronizarea firelor de executie
5. ExecutorService
6. Atomic wrappers
7. Colectii concurente

endava

FIRE DE EXECUTIE

- Un fir de executie (**thread**) este o unitate separata de executie in cadrul unui program.
- Java permite rularea mai multor threads in paralel, ceea ce se numeste **multithreading**.
- Avantaje pentru multithreading:
  - Rularea mai multe taskuri simultan (ex: procesare date + afisare interfata).
  - Imbunatatirea performantei aplicatiei
  - Evitarea blocarii programului principal atunci cand se executa operatii lente (ex: retea, fisiere, baze de date).

FIRE DE EXECUTIE



endava

CREAREA FIRELOR DE EXECUTIE

- In Java, Runnable si Thread sunt doua moduri principale de a crea si rula fire de executie (threads).
- **Thread** este o clasa care extinde clasa Object.
- Pentru a folosi Thread:
  - Creezi o subclasa a lui Thread si suprascrie metoda run().
  - Apelezi metoda start() pentru a porni firul de executie.
- **Runnable** este o interfata functionala care contine o singura metoda: run().
- Pentru a folosi Runnable:
  - se creeaza o clasa care implementeaza Runnable si supracscrie metoda run().
  - Un obiect al acestei clase este pasat unui obiect de tip Thread, care il poate rula.

endava

CREAREA FIRELOR DE EXECUTIE

Metoda din clasa Thread	Descriere scurta
`start()`	Porneste firul si apeleaza automat metoda `run()` intr-un nou context de executie.
`run()`	Contine logica firului; este apelata automat de `start()`, sau manual pentru executie secventiala.
`sleep(long millis)`	Pune firul curent in pauza pentru un numar de milisecunde.
`join()`	Asteapta ca firul respectiv sa isi termine executia inainte de a continua.
`interrupt()`	Marcheaza firul ca "interupt", intrerupand metode ca `sleep()` sau `join()`.
`getName()`	Returneaza numele firului.
`setPriority(int priority)`	Seteaza prioritatea firului (intre `Thread.MIN_PRIORITY` si `Thread.MAX_PRIORITY`).
`currentThread()` (static)	Returneaza referinta la firul care ruleaza in acel moment.

endava

CREAREA FIRELOR DE EXECUTIE

Caracteristica	Runnable	Thread
Mostenire	Poate fi folosit daca clasa mosteneste deja alta clasa	Ocupa mostenirea unei singure clase (Thread)
Flexibilitate	Mai flexibil, separa logica de executie de mecanismul de rulare	Mai putin flexibil
Recomandare	Este preferat in majoritatea cazurilor	Util in cazuri simple sau pentru testare

endava

PROBLEME IN CONTEXT MULTITHREADED

- Cand mai multe fire de executie ruleaza in paralel si acceseaza resurse comune, pot aparea o serie de probleme care duc la comportamente neasteptate, erori sau performanta slaba.

1. **Race condition:**

- Apare cand mai multe threads acceseaza si modifica simultan o resursa comuna (ex: o variabila).
- Comportamentul programului devine imprevizibil.
- Exemplu: incrementarea unei variabile partajate fara sincronizare.
- Sfaturi pentru evitare:
  - Foloseste blocuri synchronized cand accesezi resurse partajate.
  - Foloseste clase din java.util.concurrent (ex: atomic wrappers, concurrent collections).
  - Evita partajarea datelor intre threads, daca nu este necesar.

endava

PROBLEME IN CONTEXT MULTITHREADED

2. Deadlock:

- Apare cand doua sau mai multe fire asteapta resurse blocate de celelalte, formand un ciclu.
- Niciun fir nu isi poate continua executia => blocaj total.
- Exemplu: threadul A detine lock pe resursa X si asteapta pe resursa Y, iar threadul B detine lock pe resursa Y si asteapta pe resursa X.
- Sfaturi pentru evitare:
  - Pastreaza un ordin unic de obtinere a resurselor in toate firele.
  - Evita blocarea pe mai multe obiecte simultan daca se poate.

endava

PROBLEME IN CONTEXT MULTITHREADED

3. Livelock:

- Similar cu deadlock, dar firele nu sunt blocate complet — continua sa ruleze, dar nu isi finalizeaza sarcinile pentru ca se interfereaza reciproc in mod constant.
- Sfaturi pentru evitare:
  - Introdu pauze (Thread.sleep() sau yield()) in retry loops.
  - Limiteaza numarul de incercari pentru operatii repetate.

4. Starvation:

- Un thread nu reuseste sa ruleze sau sa obtina resurse pentru ca alte threads acapareaza mereu executia.
- Se intampla cand prioritizarea este inegala sau cand sincronizarea este prost gestionata.
- Sfaturi pentru evitare:
  - Nu folosi prioritati extreme (Thread.MIN_PRIORITY, Thread.MAX_PRIORITY) decat daca este absolut necesar.
  - Asigura-te ca toate threads au sansa sa ruleze (evita coduri blocate prea mult timp).

endava

SINCRONIZAREA FIRELOR DE EXECUTIE

- Cand mai multe fire de executie acceseaza resurse comune este necesara **sincronizarea**, pentru a preveni conflictele si comportamentul neasteptat. Exista mai multe moduri in care se poate face sincronizare.

1. Blocuri sincronizate

- Permite ca doar un singur fir sa execute o portiune de cod la un moment dat.
- Se obtine cu synchronized(), care cuprinde blocul de instructiuni care vrem sa se execute sincronizat si defineste un monitor comun tuturor firelor de executie pe care le sincronizam

2. **Metode sincronizate**

- Permite ca un singur fir de executie sa execute metoda sincronizata.
- Se obtine cu synchronized, definit pe metoda.
- Monitorul este this (instanta curenta).

3. Alte instrumente

- Atomic wrappers
- Concurrent collections
- Obiecte blocante: Semaphore, CountDownLatch, CyclicBarrier

endava

EXECUTOR SERVICES

- **ExecutorService** este o interfata din pachetul java.util.concurrent care gestioneaza rularea firelor de executie intr-un mod mai flexibil si mai eficient decat crearea manuala a obiectelor Thread.
- Avantaje:
  - Permite reutilizarea threadurilor prin thread pool (nu se creaza un thread nou pentru fiecare task).
  - Ofera metode pentru programarea, oprirea si monitorizarea threadurilor.
  - Elibereaza programatorul de detaliile tehnice legate de gestionarea threadurilor.

endava

EXECUTOR SERVICES

🔁 Tipuri de thread pool-uri

Metoda	Descriere
`newFixedThreadPool(n)`	Pool cu un numar fix de fire
`newCachedThreadPool()`	Pool cu numar variabil de fire (pentru sarcini scurte/frecvente)
`newSingleThreadExecutor()`	Un singur fir (executa sarcini secvential)
`newScheduledThreadPool(n)`	Pool care permite executie intarziata sau periodica

endava

EXECUTOR SERVICES

📥 Metode importante

Metoda	Descriere
`submit(Runnable)`	Trimite o sarcina fara rezultat (`Runnable`)
`submit(Callable)`	Trimite o sarcina care returneaza un rezultat
`invokeAll(...)`	Ruleaza mai multe sarcini si asteapta toate rezultatele
`shutdown()`	Inchide executorul, fara a intrerupe sarcinile active
`shutdownNow()`	Incearca sa opreasca toate sarcinile imediat
`isShutdown()`	Verifica daca a fost inchis
`isTerminated()`	Verifica daca toate sarcinile s-au terminat

endava

ATOMIC WRAPPERS

- **Atomic wrappers** sunt clase din pachetul java.util.concurrent.atomic care ofera operatii thread-safe (atomice) pe variabile primitive sau obiecte fara a folosi synchronized.
- Avantaje
  - Evitarea race condition.
  - Ofera performanta mai buna decat synchronized, mai ales in operatii simple (ex: increment).
- Limitari:
  - Nu pot fi folosite pentru operatii complexe (ex: mai multi pasi care trebuie sa fie atomici impreuna).

Clasa	Descriere
`AtomicInteger`	Operatii atomice pe `int`
`AtomicLong`	Operatii atomice pe `long`
`AtomicBoolean`	Operatii atomice pe `boolean`
`AtomicReference<T>`	Operatii atomice pe obiecte generice
`AtomicIntegerArray`	Operatii atomice pe tablouri de int

CONCURRENT COLLECTIONS

- **Colectiile concurente** sunt clase din java.util.concurrent proiectate pentru a fi thread-safe.
- Avantaje:
  - Nu mai trebuie sa sincronizezi manual accesul.
  - Mai rapide si mai scalabile decat solutiile bazate pe synchronized.
  - Unele sunt lock-free, altele folosesc blocare controlata intern.
- Cand sa nu le folosesti:
  - Daca ai nevoie de operatii complexe pe mai multe elemente care trebuie sa fie toate atomice (in acest caz, foloseste synchronized).
  - Daca aplicatia este strict single-threaded – nu are rost sa folosesti colectii concurente.

CONCURRENT COLLECTIONS

Clasa	Descriere
`ConcurrentHashMap`	Varianta thread-safe si rapida a `HashMap`
`CopyOnWriteArrayList`	Lista thread-safe pentru citiri frecvente si scrieri rare
`CopyOnWriteArraySet`	Set thread-safe bazat pe `CopyOnWriteArrayList`
`ConcurrentLinkedQueue`	Coada nelimitata non-blocking (FIFO), bazata pe lock-free algoritm
`ConcurrentLinkedDeque`	Deque thread-safe, non-blocking
`BlockingQueue` (interfata)	Coada care **asteapta** cand adaugi sau extragi elemente
`ArrayBlockingQueue`	Coada cu capacitate fixa, blocanta
`LinkedBlockingQueue`	Coada cu capacitate optionala, blocanta
`PriorityBlockingQueue`	Coada cu prioritizare, thread-safe

endava

Q&A

THANK YOU

DANIELA SPILCĂ